

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Prudhvi raj.b

Annexure A

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.
2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.
3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.
4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.
5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer

Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. K-Nearest Neighbour (KNN) – Euclidean Distance

```
import math
```

```
# Training dataset
```

```
data = [  
    ([1, 2], "A"),  
    ([2, 3], "A"),  
    ([3, 3], "A"),  
    ([6, 5], "B"),  
    ([7, 7], "B"),  
    ([8, 6], "B")  
]
```

```
# Test data
```

```
test_data = [  
    ([2, 2], "A"),  
    ([7, 6], "B")  
]
```

```
k = int(input("Enter value of K: "))
```

```
def distance(p1, p2):  
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
```

```
correct = 0
```

```
for test_point, actual in test_data:
```

```
    distances = []
```

```
    for point, label in data:
```

```
        d = distance(test_point, point)
```

```
        distances.append((d, label))
```

```
    distances.sort()
```

```
    neighbors = distances[:k]
```

```
    votes = {}
```

```
    for d, label in neighbors:
```

```
        votes[label] = votes.get(label, 0) + 1
```

```
    predicted = max(votes, key=votes.get)
```

```

print("\nTest Point:", test_point)
print("Predicted Class:", predicted)
print("Actual Class:", actual)

if predicted == actual:
    correct += 1

accuracy = correct / len(test_data) * 100

print("\nAccuracy:", accuracy, "%")

```

OUTPUT:

```

Enter value of K: 2

Test Point: [2, 2]
Predicted Class: A
Actual Class: A

Test Point: [7, 6]
Predicted Class: B
Actual Class: B

Accuracy: 100.0 %

```

2. Locally Weighted Regression

```

import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
y = np.array([2, 4, 5, 4, 6, 7, 8, 9], dtype=float)

# Query point
query = float(input("Enter query point: "))

# Bandwidth
tau = 1.0

# Calculate Gaussian weights
weights = np.exp(-(X - query) ** 2 / (2 * tau ** 2))

# Weighted Linear Regression
X_matrix = np.vstack((np.ones(len(X)), X)).T
W = np.diag(weights)

```

```

theta = np.linalg.pinv(
    X_matrix.T @ W @ X_matrix
) @ X_matrix.T @ W @ y

lwr_prediction = theta[0] + theta[1] * query

# Standard Linear Regression
standard_theta = np.polyfit(X, y, 1)
standard_prediction = standard_theta[0] * query + standard_theta[1]

print("\nLWR Prediction:", round(lwr_prediction, 2))
print("Linear Regression Prediction:",
      round(standard_prediction, 2))

# Plot
x_plot = np.linspace(1, 8, 100)

lwr_values = []
for x in x_plot:
    w = np.exp(-((X - x) ** 2) / (2 * tau ** 2))
    W = np.diag(w)

    theta = np.linalg.pinv(
        X_matrix.T @ W @ X_matrix
    ) @ X_matrix.T @ W @ y

    lwr_values.append(theta[0] + theta[1] * x)

plt.scatter(X, y, label="Data")
plt.plot(x_plot, lwr_values, label="LWR")
plt.plot(
    x_plot,
    standard_theta[0] * x_plot + standard_theta[1],
    label="Linear Regression"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()

```

OUTPUT:

```
Enter query point: 2
LWR Prediction: 3.61
Linear Regression Prediction: 3.33
/home/main.py:61: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

3. Radial Basis Function (RBF) Network

import numpy as np

Dataset

X = np.array([1, 2, 3, 4, 5, 6], dtype=float)

y = np.array([2, 4, 6, 8, 10, 12], dtype=float)

RBF centers

centers = np.array([2, 4, 6], dtype=float)

Width

sigma = 1.0

Gaussian RBF function

def rbf(x, center):

return np.exp(-(x - center) ** 2) / (2 * sigma ** 2))

Create RBF feature matrix

Phi = np.zeros((len(X), len(centers)))

for i in range(len(X)):

for j in range(len(centers)):

Phi[i, j] = rbf(X[i], centers[j])

Train output weights

weights = np.linalg.pinv(Phi) @ y

Prediction

predictions = Phi @ weights

print("Actual Outputs:")

print(y)

print("\nPredicted Outputs:")

print(np.round(predictions, 2))

Predict new value

```
new_x = float(input("\nEnter value to predict: "))
```

```
new_phi = np.array([
    rbf(new_x, center) for center in centers
])
```

```
prediction = new_phi @ weights
```

```
print("Predicted Output:", round(prediction, 2))
```

OUTPUT

```
Actual Outputs:
[ 2.  4.  6.  8. 10. 12.]

Predicted Outputs:
[ 2.06  4.11  5.76  7.94 10.35 11.8 ]

Enter value to predict: 5
Predicted Output: 10.35
```

4. Case-Based Learning

```
import math
```

```
# Past cases
```

```
cases = [
    ([25, 30000, 2], "Low Risk"),
    ([30, 40000, 3], "Low Risk"),
    ([35, 60000, 5], "Medium Risk"),
    ([40, 70000, 6], "Medium Risk"),
    ([50, 90000, 8], "High Risk"),
    ([55, 100000, 9], "High Risk")
]
```

```
# New query
```

```
query = [38, 65000, 6]
```

```
# Normalize values for fair distance calculation
```

```
features = list(zip(*[case[0] for case in cases]))
```

```
mins = [min(f) for f in features]
```

```
maxs = [max(f) for f in features]
```

```
def normalize(value, index):
```

```
    if maxs[index] == mins[index]:
```

```
        return 0
```

```
    return (value - mins[index]) / (maxs[index] - mins[index])
```

```

query_normalized = [
    normalize(query[i], i)
    for i in range(len(query))
]

best_distance = float("inf")
best_case = None
best_solution = None

for case, solution in cases:

    case_normalized = [
        normalize(case[i], i)
        for i in range(len(case))
    ]

    distance = math.sqrt(
        sum(
            (query_normalized[i] - case_normalized[i]) ** 2
            for i in range(len(query))
        )
    )

    print("Case:", case,
          "Solution:", solution,
          "Distance:", round(distance, 3))

    if distance < best_distance:
        best_distance = distance
        best_case = case
        best_solution = solution

print("\nNew Query:", query)
print("Most Similar Case:", best_case)
print("Distance:", round(best_distance, 3))
print("Predicted Solution:", best_solution)

```

OUTPUT:

```

Case: [25, 30000, 2] Solution: Low Risk Distance: 0.874
Case: [30, 40000, 3] Solution: Low Risk Distance: 0.618
Case: [35, 60000, 5] Solution: Medium Risk Distance: 0.188
Case: [40, 70000, 6] Solution: Medium Risk Distance: 0.098
Case: [50, 90000, 8] Solution: High Risk Distance: 0.608
Case: [55, 100000, 9] Solution: High Risk Distance: 0.869

New Query: [38, 65000, 6]
Most Similar Case: [40, 70000, 6]
Distance: 0.098
Predicted Solution: Medium Risk

```

5. K-Means Clustering

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Dataset
```

```
X = np.array([
```

```
    [1, 2],
```

```
    [2, 1],
```

```
    [2, 3],
```

```
    [3, 2],
```

```
    [8, 8],
```

```
    [9, 7],
```

```
    [8, 9],
```

```
    [9, 9]
```

```
], dtype=float)
```

```
k = int(input("Enter number of clusters: "))
```

```
# Initialize centroids using first k points
```

```
centroids = X[:k].copy()
```

```
for iteration in range(100):
```

```
    # Assign points to nearest centroid
```

```
    clusters = []
```

```
    for point in X:
```

```
        distances = [
```

```
            np.linalg.norm(point - centroid)
```

```
            for centroid in centroids
```

```
        ]
```

```

        clusters.append(np.argmin(distances))

clusters = np.array(clusters)

# Update centroids
new_centroids = []

for i in range(k):

    points = X[clusters == i]

    if len(points) > 0:
        new_centroids.append(points.mean(axis=0))
    else:
        new_centroids.append(centroids[i])

new_centroids = np.array(new_centroids)

# Check convergence
if np.allclose(centroids, new_centroids):
    break

centroids = new_centroids

print("\nFinal Centroids:")
print(centroids)

print("\nClusters:")

for i in range(k):

    print("\nCluster", i + 1)

    for point in X[clusters == i]:
        print(point)

# Visualization
for i in range(k):
    points = X[clusters == i]
    plt.scatter(
        points[:, 0],
        points[:, 1],
        label="Cluster " + str(i + 1)
    )

```

```
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=200,
    label="Centroids"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

OUTPUT:

```
Enter number of clusters: 4

Final Centroids:
[[1.5  2.5 ]
 [2.5  1.5 ]
 [8.5  8.25]
 [6.   4.5 ]]

Clusters:

Cluster 1
[1. 2.]
[2. 3.]

Cluster 2
[2. 1.]
[3. 2.]

Cluster 3
[8. 8.]
[9. 7.]
[8. 9.]
[9. 9.]

Cluster 4
/home/main.py:90: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```